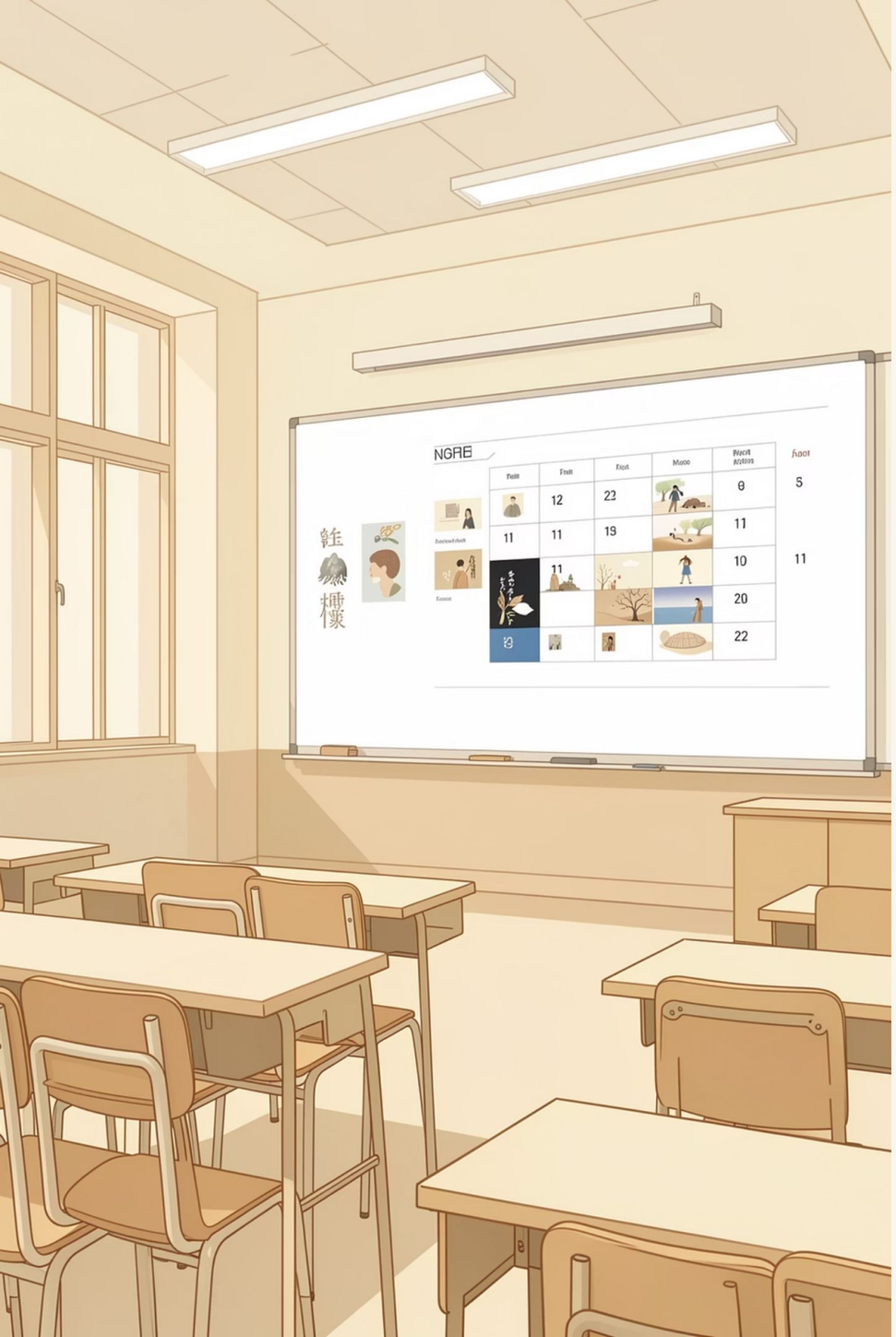

Faculty Timetable Generator

Automated scheduling software that transforms chaotic faculty assignments into organized, conflict-free timetables—solving a real college challenge with professional-grade C programming.

The Problem

Faculty timetable generation solves several real-world challenges:

- Manual scheduling chaos
- Time consumption
- Conflicts
- Uneven workloads

Creating timetables by hand consumes hours of administrative time. Coordinators juggle faculty availability, room assignments, lab sessions, and conflicting requests—all while trying to maintain fairness and balance. The result? Schedules riddled with errors, uneven workloads, and frustrated faculty members.

Why This Project Matters

This project stands out as a portfolio piece because it combines practical problem-solving with solid programming fundamentals:

1

Solves Real College Problems

Addresses actual pain point in academic administration

2

Demonstrates Core Programming Skills

Uses arrays, structures, file handling, and algorithmic logic

3

Professional Appearance

Produces clean, organized output like commercial software

4

Admin-Level Complexity

Handles real-world constraints and requirements

Our Solution

Our Faculty Timetable Generator addresses these challenges with three core features that ensure efficient, fair, and conflict-free scheduling:

Zero Conflicts

No professor teaching two classes simultaneously

Fair Distribution

Equal lecture loads across all faculty

Lab Integration

Seamless handling of theory and practical sessions

Key Features

The system includes three powerful features that work together to create optimal timetables:

Smart Load Balancing

Distributes teaching loads proportionally across all instructors

Lab + Theory Management

Handles both specialized lab requirements and standard lecture periods

Export to Text File

Generates portable, shareable timetables with day-wise schedules, room assignments, and conflict-free verification

Technical Implementation

Built with C programming fundamentals, this project demonstrates three key technical components:

Data Structures

Uses structs to organize faculty data, courses, time slots, and room assignments

Nested Loops

Multiple iteration strategies to check conflicts and optimize scheduling

File I/O

Reads input requirements and writes formatted timetable output

How It Works

The system follows a four-step intelligent process to generate conflict-free timetables:

Input Collection

Gather faculty availability, course requirements, room capacities

Conflict Analysis

System identifies overlapping time slots

Resolution Engine

Algorithm rearranges assignments

Validation Report

Generate comprehensive report with no conflicts

```
========== FINAL TIMETABLE ==========

Mon:
  Slot 1: v (v)
  Slot 2: s (s)
  Slot 3: a (a)
  Slot 4: FREE

Tue:
  Slot 1: FREE
  Slot 2: FREE
  Slot 3: FREE
  Slot 4: FREE

Wed:
  Slot 1: FREE
  Slot 2: FREE
  Slot 3: FREE
  Slot 4: FREE

Thu:
  Slot 1: FREE
  Slot 2: FREE
  Slot 3: FREE
  Slot 4: FREE

Fri:
  Slot 1: FREE
```

```
Enter faculty 3 name: a
Enter subject taught by a: a

Enter availability for a (1 = FREE, 0 = BUSY
    )
Mon:
  Slot 1: 0
  Slot 2: 1
  Slot 3: 1
  Slot 4: 1
Tue:
  Slot 1: 0
  Slot 2: 0
  Slot 3: 1
  Slot 4: 1
Wed:
  Slot 1: 1
  Slot 2: 0
  Slot 3: 1
  Slot 4: 1
Thu:
  Slot 1: 1
  Slot 2: 0
  Slot 3: 1
  Slot 4: 1
Fri:
  Slot 1: 1
  Slot 2: 1
  Slot 3: 1
```

```
Enter faculty 2 name: s
Enter subject taught by s: s

Enter availability for s (1 = FREE, 0 = BUSY
    )
Mon:
  Slot 1: 0
  Slot 2: 1
  Slot 3: 1
  Slot 4: 1
Tue:
  Slot 1: 1
  Slot 2: 1
  Slot 3: 1
  Slot 4: 1
Wed:
  Slot 1: 1
  Slot 2: 1
  Slot 3: 1
  Slot 4: 1
Thu:
  Slot 1: 1
  Slot 2: 0
  Slot 3: 1
  Slot 4: 1
Fri:
  Slot 1: 1
  Slot 2: 1
  Slot 3: 1
```

```
Enter faculty 1 name: v
Enter subject taught by v: v

Enter availability for v (1 = FREE, 0 = BUSY
    )
Mon:
  Slot 1: 1
  Slot 2: 1
  Slot 3: 0
  Slot 4: 1
Tue:
  Slot 1: 1
  Slot 2: 0
  Slot 3: 1
  Slot 4: 1
Wed:
  Slot 1: 0
  Slot 2: 1
  Slot 3: 1
  Slot 4: 1
Thu:
  Slot 1: 1
  Slot 2: 1
  Slot 3: 1
  Slot 4: 0
Fri:
  Slot 1: 1
  Slot 2: 0
```

```
  Slot 4: FREE

Tue:
  Slot 1: FREE
  Slot 2: FREE
  Slot 3: FREE
  Slot 4: FREE

Wed:
  Slot 1: FREE
  Slot 2: FREE
  Slot 3: FREE
  Slot 4: FREE

Thu:
  Slot 1: FREE
  Slot 2: FREE
  Slot 3: FREE
  Slot 4: FREE

Fri:
  Slot 1: FREE
  Slot 2: FREE
  Slot 3: FREE
  Slot 4: FREE
```

https://www.programiz.com/online-compiler/3GwSLgIDPNdSM

Made with GAMMA

Conclusion

- Complete, well-documented C source code with all core features implemented and tested
- Showcases understanding of real-world constraints and professional software development practices
- Ready to extend with GUI interface, spreadsheet import, or integration with existing college management systems
- Demonstrates problem-solving skills that impress both professors and future employers in tech